UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 03

Sistemas de Numeración

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 ¿Por qué contamos en base 10? 2

2 Otros sistemas a lo largo de la historia 2
2.1 Los mayas: base 20 2
2.2 Los babilonios: base 60 2
2.3 La propuesta de la base 12 3

3 ¿Por qué las computadoras usan base 2? 3

4 Sistema binario (base 2) 4
4.1 Binario → Decimal 4
4.2 Decimal → Binario 4

5 Base 8 — Octal 5
5.1 Octal ↔ Binario: la relación directa 5

6 Base 16 — Hexadecimal 5
6.1 ¿Por qué existe el hexadecimal? 6

7 Resumen de conversiones 6

8 Ejercicios 6

9 Resumen 7

¿Por qué contamos en base 10?

La respuesta corta: porque tenemos **diez dedos**.

Contar en base 10 (o *sistema decimal*) significa que usamos diez símbolos distintos — 0, 1, 2, . . . , 9 — y que cada posición en un número vale **diez veces** la posición de su derecha.

$$9{,}324_{10} = 9 \times 10^3 + 3 \times 10^2 + 2 \times 10^1 + 4 \times 10^0 = 9{,}000 + 300 + 20 + 4$$

La posición más a la derecha vale base$^0 = 1$, la siguiente base^1, luego base^2, y así sucesivamente. Este patrón funciona **igual en cualquier base**; solo cambia el número de símbolos disponibles y el peso de cada posición.

✔ Recuerda

Un sistema de numeración **posicional** es aquel donde el valor de un dígito depende de la posición que ocupa, no solo del símbolo en sí. El sistema romano (IV, IX. . .) *no* es estrictamente posicional; el nuestro, sí.

Otros sistemas a lo largo de la historia

Los humanos no siempre contamos de diez en diez. Distintas civilizaciones eligieron bases diferentes según su anatomía, su astronomía o su necesidad de dividir cosas en partes iguales.

• Los mayas: base 20

Los mayas usaban un sistema **vigesimal** (base 20). La razón también es corporal: en lugar de contar solo con los dedos de las manos, contaban además los **dedos de los pies**. Veinte extremidades, veinte símbolos.

Su sistema tenía tres símbolos básicos: un punto (1), una raya (5) y un caracol (0). Con ellos construían valores hasta 19 y luego cambiaban de posición, igual que nosotros al llegar a 9.

El sistema maya fue uno de los primeros en incluir el **cero** como valor posicional, probablemente desde el siglo IV d.C., siglos antes de que el concepto llegara a Europa. Para conocer más: Ifrah, G. *Historia Universal de las Cifras*. Espasa Calpe, 1994.

• Los babilonios: base 60

¿Por qué un minuto tiene 60 segundos? ¿Por qué una hora tiene 60 minutos? ¿Por qué un círculo tiene 360 grados?

Todo eso viene de los **sumerios y babilonios**, que hace más de 4 000 años usaban un sistema **sexagesimal** (base 60).

La elección no fue arbitraria: **60 es extraordinariamente divisible**. Sus divisores son 1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30 y 60. Eso significa que $\frac{1}{3}$, $\frac{1}{4}$ y $\frac{1}{5}$ de una hora

resultan en números enteros de minutos, algo muy útil para repartir, medir campos o calcular calendarios.

> Los griegos heredaron la astronomía babilónica y con ella sus unidades. Ptolomeo dividió el círculo en 360 partes (grados), cada grado en 60 *partes primeras* (minutos) y cada minuto en 60 *partes segundas* (segundos). Esos nombres llegan hasta hoy.
> *Referencia:* Neugebauer, O. *The Exact Sciences in Antiquity*. Dover, 1969.

• La propuesta de la base 12

Durante la Revolución Francesa (finales del siglo XVIII), mientras se diseñaba el sistema métrico decimal, varios matemáticos argumentaron que sería más práctico adoptar la **base 12** (duodecimal) en lugar de la base 10.

El argumento: **12 tiene más divisores enteros que 10**.

Base	Divisores	Fracciones exactas
10	1, 2, 5, 10	$\frac{1}{2}$, $\frac{1}{5}$
12	1, 2, 3, 4, 6, 12	$\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{4}$, $\frac{1}{6}$

Con base 12, un tercio o un cuarto serían tan “limpios” como la mitad lo es hoy. Hoy en día la *Dozenal Society of Great Britain* y la *Dozenal Society of America* siguen promoviendo esta idea.

> Curiosamente, ya usamos la base 12 en la vida cotidiana: 12 meses, 12 horas en el reloj, una docena de huevos, una gruesa (144 = 12^2). La base 12 nunca desapareció del todo.
> *Referencia:* `https://www.dozenalsociety.org.uk`

¿Por qué las computadoras usan base 2?

Una computadora está construida con millones de pequeños interruptores electrónicos llamados **transistores**. Cada transistor solo puede estar en uno de dos estados: **encendido** o **apagado**.

Por eso la base natural de cualquier computadora es la **base 2**: dos estados, dos símbolos (0 y 1), llamados **bits** (*binary digits*).

> ✔ **Recuerda**
>
> **¿Por qué no base 10?** Si quisiéramos representar diez estados distintos con voltaje eléctrico (0 V, 1 V, 2 V, . . . , 9 V), cualquier ruido o variación mínima podría confundir un 3 con un 4. Con solo dos estados (bajo / alto), el circuito es **mucho más robusto y rápido** de fabricar y operar.

- **1 bit** — un dígito binario: 0 ó 1.
- **4 bits** — un *nibble*: puede representar 0–15.
- **8 bits** — un *byte*: puede representar 0–255.
- **32** / **64 bits** — tamaños típicos de las palabras (*words*) que maneja el procesador de tu computadora.

Sistema binario (base 2)

En binario solo existen los dígitos **0** y **1**. Cada posición vale el **doble** de la posición a su derecha (potencias de 2).

Posición	7	6	5	4	3	2	1	0
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1

- **Binario → Decimal**

  Multiplica cada bit por su peso posicional y suma.

$$1001_2 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 0 + 0 + 1 = 9_{10}$$

$$1100\,1000_2 = 2^7 + 2^6 + 2^3 = 128 + 64 + 8 = 200_{10}$$

Truco: ignora los ceros; solo suma los pesos donde aparece un 1.

- **Decimal → Binario**

  Divide repetidamente entre 2 y anota los restos. Los restos, leídos **de abajo hacia arriba**, forman el número binario.

25 ÷ 2 = 12	resto **1**
12 ÷ 2 = 6	resto **0**
6 ÷ 2 = 3	resto **0**
3 ÷ 2 = 1	resto **1**
1 ÷ 2 = 0	resto **1**

Leyendo los restos de abajo hacia arriba: $25_{10} = 11001_2$.

Verificación: $16 + 8 + 1 = 25$. ✓

Las divisiones sucesivas dan: $1521_{10} = 10111110001_2$.
Verificación: $1024 + 256 + 128 + 64 + 32 + 16 + 1 = 1521$. ✓

Base 8 — Octal

El sistema octal usa ocho símbolos: **0, 1, 2, 3, 4, 5, 6, 7**. Cada posición vale el **óctuple** de la posición a su derecha (potencias de 8).

$$4702_8 = 4 \times 8^3 + 7 \times 8^2 + 0 \times 8^1 + 2 \times 8^0 = 2048 + 448 + 0 + 2 = 2498_{10}$$

2498 ÷ 8 = 312 resto **2**
312 ÷ 8 = 39 resto **0**
39 ÷ 8 = 4 resto **7**
4 ÷ 8 = 0 resto **4**

$2498_{10} = 4702_8$. ✓

• Octal ↔ Binario: la relación directa

3 bits representan exactamente los valores 0–7. Por eso convertir entre octal y binario es inmediato: sustituye cada dígito octal por su representación de **3 bits**.

Octal	3 bits
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

$$4702_8 \rightarrow \underbrace{100}_{4}\ \underbrace{111}_{7}\ \underbrace{000}_{0}\ \underbrace{010}_{2} = 100\,111\,000\,010_2$$

Base 16 — Hexadecimal

El hexadecimal usa **dieciséis** símbolos. Como solo tenemos diez cifras (0–9), se completa con letras: A=10, B=11, C=12, D=13, E=14, F=15.

Decimal	Hex	Decimal	Hex
0	0	8	8
1	1	9	9
2	2	10	A
3	3	11	B
4	4	12	C
5	5	13	D
6	6	14	E
7	7	15	F

- **¿Por qué existe el hexadecimal?**

Porque escribir números binarios largos es **tedioso y propenso a errores**. Un byte son 8 bits; un entero de 32 bits son 32 ceros y unos. El hexadecimal es un **atajo** para representar binario de forma compacta.

La clave: **4 bits = 1 dígito hexadecimal** exactamente (pues $2^4 = 16$).

$$\texttt{EA23}_{16} \rightarrow \underbrace{1110}_{E}\ \underbrace{1010}_{A}\ \underbrace{0010}_{2}\ \underbrace{0011}_{3} = 1110\,1010\,0010\,0011_2$$

Sustituye cada dígito hex por su grupo de **4 bits**. Directo.

$$0111\,0000\,1001\,0011\,1100\,1011_2 \rightarrow \underbrace{0111}_{7}\ \underbrace{0000}_{0}\ \underbrace{1001}_{9}\ \underbrace{0011}_{3}\ \underbrace{1100}_{C}\ \underbrace{1011}_{B} = 7093\texttt{CB}_{16}$$

Agrupa de derecha a izquierda en bloques de 4 bits y reemplaza.

En programación, el hexadecimal se escribe con los prefijos `0x` (C, C++, Python) o `#` (colores en CSS/HTML). Por ejemplo, `0xFF` es 255_{10}, y el color blanco en HTML es `#FFFFFF`.

Resumen de conversiones

Conversión	Método	Truco rápido
Cualquier base $\rightarrow$ decimal	Suma pesos posicionales	$\sum d_i \times b^i$
Decimal $\rightarrow$ cualquier base	Divisiones sucesivas por b	Leer restos de abajo arriba
Binario $\leftrightarrow$ octal	Grupos de 3 bits	Directo, sin pasar por decimal
Binario $\leftrightarrow$ hexadecimal	Grupos de 4 bits	Directo, sin pasar por decimal

Ejercicios

1. Convierte $13{,}333_{10}$ a **binario**.
2. Convierte $1001\,0110\,0100\,1100_2$ a **decimal**.

3. Convierte $1111\,0\,1010\,1100\,0\,1010\,1011_2$ a **hexadecimal**.
4. Convierte $\mathtt{3F2A1}_{16}$ a **binario**.
5. Convierte 31037562_8 a **hexadecimal**.
6. Convierte $12300\,2130_4$ a **hexadecimal**.

Para los ejercicios 5 y 6 lo más sencillo es pasar primero a binario y luego agrupar en bloques de 4. Evita pasar por decimal si puedes.

Problema de concurso

Carina está escribiendo todos los números de uno en uno en su calculadora, empezando en 1. Pero a la calculadora se le rompieron las teclas del 2 y del 7, por lo que cualquier número que lleve esos dígitos se lo salta. ¿Cuál es el milésimo número que escribirá Carina?

Si Carina no puede usar los dígitos 2 ni 7, en realidad está contando en un sistema de **ocho símbolos** (0, 1, 3, 4, 5, 6, 8, 9). ¿Qué relación tiene eso con una base que ya conoces?

Resumen

✔ **Recuerda**

- Contamos en base 10 por razones anatómicas; otras civilizaciones eligieron bases distintas (20 mayas, 60 babilónicos).
- Las computadoras usan base 2 porque los transistores tienen dos estados: encendido/apagado.
- **Binario** $\rightarrow$ **decimal:** suma los pesos posicionales donde hay un 1.
- **Decimal** $\rightarrow$ **base** b**:** divide repetidamente entre b y lee los restos de abajo hacia arriba.
- **Binario** $\leftrightarrow$ **octal:** grupos de 3 bits. **Binario** $\leftrightarrow$ **hex:** grupos de 4 bits.
- Hexadecimal existe para escribir binario de forma compacta; es la notación más usada en programación de bajo nivel.